

The step_7 Architecture

How the Solanus Casey archival tool is built

David Falk
Solanus Project Pipeline

Contents

1	TL;DR	2
2	Key points (the five design commitments)	2
3	The corpus we start from	2
4	The diff-and-rerun pipeline	3
4.1	The intuition: a recipe that re-cooks only what changed	3
4.2	A Stage declares four things	3
4.3	The fingerprint: four ingredients, any one of which makes a stage stale	4
4.4	What “stale” means, and the one magic line	4
4.5	Two guardrails	4
4.6	Driving it	5
5	The model-agnostic three-axis design	5
5.1	The pattern (the same for all three)	5
5.2	The embedding axis has one extra twist: partitioned spaces	6
6	Always-on cost logging	6
7	Provenance and archival standards	7
8	The tiers (and the stages that fill them)	7
9	Where the axes and tiers meet the user: the dev app	8
10	Recommendations: how to extend	9
11	Caveats	9
12	Sources	9

1 TL;DR

The whole machine in six sentences

- **One DAG, re-run only what changed.** Every unit of work is a content-hashed **Stage**; a stage is *stale* if its inputs, its code, the config, or any upstream stage changed, and `run()` re-executes the stale stages *and everything downstream*.
- **Three swappable model axes.** Generation **LLM**, **embedding** space, and **reranker** are config variables behind thin adapters — swapping one is an edit to a dict, never a rewrite of the code that uses it.
- **Every model call is cost-logged, always.** Nothing calls a provider directly; one funnel (`lib/costlog.py`) records every billed call, so the dollar ledger is complete and the free path stays free.
- **Provenance is first-class.** Every chunk, fact, and answer cites `doc_id+rid+page+vertices` — the exact polygon on the exact scanned page.
- **Non-destructive, always.** Stages only write new artifacts; the runner never deletes; `step_6/` source data is read-only.
- **Tiered, so you choose your altitude.** Tier 0 solid RAG → Tier 1 structural truth → Tier 2 knowledge graph → Tier 3 multimodal → Tier 4 scholarly platform.

2 Key points (the five design commitments)

Everything below is a consequence of five commitments. They are not slogans — each one is enforced by a specific piece of code you can open and read.

The five commitments

1. **Diff-and-rerun reproducibility** — a content-hashed stage DAG; staleness propagates. (`lib/pipeline.py`)
2. **Always log prices** — one funnel records every billed call; the free path is free. (`lib/costlog.py`)
3. **Model-agnostic, three axes** — LLM / embedding / reranker are config variables behind adapters. (`config.py`, `lib/providers/`)
4. **Provenance-first + archival standards** — cite `doc_id+rid+page+vertices`; graph aligned to RiC-O; dates in EDTF.
5. **Expandable** — add a stage, an embedding space, a model, or a tool without touching the rest.

3 The corpus we start from

Before any of the machinery makes sense, you need to see the shape of the raw material, because the architecture is built *around* that shape. `step_6` hands us two read-only JSON files.

documents.json — **570 letters**. Each record carries an `id` (e.g. `Volume_1__p001`), a `recipient`, a `date` (e.g. “Aug. 23rd, 1896”), and — the useful part — a `text_by_label` dict that *already separates* the letter into labelled fields: `src_content`, `src_recipient`, `src_greeting`, `src_date`, `src_signature`, `archv_date`, and so on. It also carries `regions[]`, where each region has a `rid`, a `category`, the OCR `text`, an OCR `min_conf`, and the gold polygon vertices (pixel geometry on the page).

notebooks.json — **716 notebook pages**. These are Fr. Solanus’s running register of favours. Each page has a `notebook` name (e.g. “FR. SOLANUS, NOTEBOOK NO. 1.”), a `pdf_page_number`, an archival `archv_date`, and an `entries[]` list. Each *entry* carries its `rid`, its `text`, a `date` (often just “30” or “Mar. 30”), a `page_label` (literally “Page 1 Cont.” for spill-over pages), and `linked` (degree-1 links from the gold connection graph).

Two features of this shape drive the entire design: **labelled fields you can read off for free**, and **per-region geometry you can cite**. `lib/chunks.py` turns the two files into **~9,342 retrieval chunks** (one per letter, one per notebook entry), each carrying its provenance metadata so any hit can point back at the exact region it came from.

A concrete entry that explains why half the pipeline exists

The notebook entry `doc_1.src_content.4` stores its date as the bare string “30” and is linked to `doc_1.src_content.10`; the page above it is labelled “Page 1 Cont.”. On its own, that entry — no year, half a thought, continued from somewhere else — is almost invisible to a search engine. The structural-truth stages (§8) exist precisely to rebuild the *whole* favour, with its date, before anything tries to retrieve it.

4 The diff-and-rerun pipeline

This is the single most important idea in `step_7`, so we build it up slowly.

4.1 The intuition: a recipe that re-cooks only what changed

Picture the whole build as a recipe where each step depends on earlier ones: chop the vegetables, then sauté them, then simmer. If you change one early ingredient, you want *only* the affected later steps to re-cook — not the entire meal (wasteful), and not nothing (you’d serve stale food). That is exactly what the pipeline does, automatically, for a graph of build steps.

4.2 A Stage declares four things

Each unit of work is a `Stage` that names its dependencies, what it reads, and what it writes:

```
Stage(name = "embed_corpus",
      fn = embed_corpus.run, # the function that does the work
      deps = [...], # upstream stage names it depends on
      inputs = [DOCS, NB, ".../embed.py"], # path globs it READS
      outputs = [str(DATA / "vectors")] # path globs it WRITES
```

4.3 The fingerprint: four ingredients, any one of which makes a stage stale

A stage’s **fingerprint** is a SHA-256 hash over four things. The cleverness is in *what* goes into the hash — it is chosen so that the four ways a result can become out-of-date all map to “the hash changed.”

1. **The source code of `fn()`** (via `inspect.getsource`) — so editing the stage’s logic re-runs it.
2. **The config fingerprint** (`config.fingerprint()`) — so changing a model, a price, or the embedding matrix re-runs the stages that depend on config.
3. **The contents of every input file** — so refining the upstream data re-runs the stage.
4. **The fingerprints of all upstream stages** — so staleness *cascades* downstream automatically.

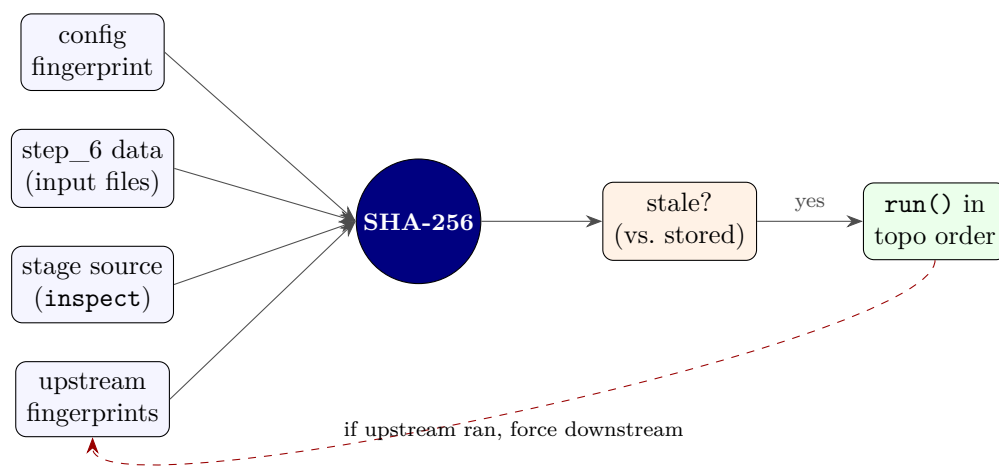


Figure 1: The fingerprint funnel. Any of the four inputs changing flips the stage to *stale*; running a stage forces everything downstream of it to run too (red loop) — which is how a fix to an early artifact “populates all the way up.”

4.4 What “stale” means, and the one magic line

A stage is **stale** when its fingerprint differs from the stored one (in `_pipeline_state.json`), *or* an output file is missing, *or* an upstream stage re-ran this pass. `status()` prints the diff first (read-only); `run()` executes stale stages in topological order and — this is the magic line — **forces everything downstream of anything that runs**. So if you refine the cleaned OCR text, the rerun naturally flows entities → resolution → graph → embeddings → indexes, “populating all the way up” with no manual bookkeeping.

4.5 Two guardrails

- **Non-destructive.** Stages only write *new* artifacts; the runner *never deletes*. `step_6/` source data is never touched — you can always recover.
- **Stdlib only.** The engine is just hashing, `glob`, and a JSON state file — zero dependencies — so the reproducibility backbone can never rot because a library moved.

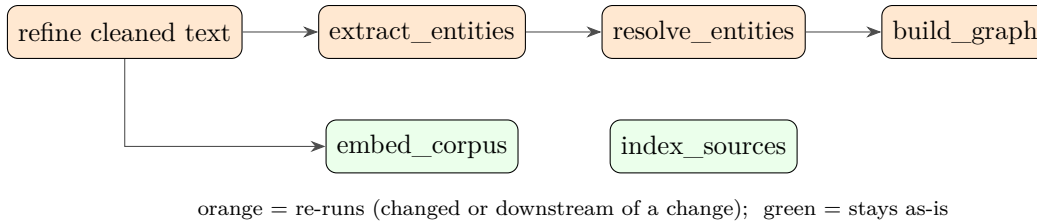


Figure 2: Editing one upstream artifact (left) marks it stale and cascades to everything that depends on it; unrelated branches stay fresh and are skipped.

4.6 Driving it

You declare the DAG and drive it from `run.py`:

```
python run.py status # show the diff: what's stale and WHY (read-only)
python run.py run # run all stale stages + everything downstream
python run.py run --only embed_corpus
python run.py run --dry # report what WOULD run; execute nothing
python run.py graph # print the dependency DAG
```

Nothing runs on import, and unimplemented stages raise `NotImplementedError` — so the DAG is fully *inspectable* long before it is fully *runnable*. That is deliberate: David runs paid stages explicitly, when ready.

5 The model-agnostic three-axis design

A RAG system plugs a model into exactly three places. `step_7` treats each as a **variable you select**, not a dependency you hard-wire.

Axis	What it does	Where it lives	Adapter
LLM	reads context, writes the cited answer, drives tool-use	<code>config.LLMS</code>	<code>providers/llm.py</code>
Embedding	turns text into a vector for similarity search	<code>config.EMBEDDINGS</code>	<code>providers/embed.py</code>
Reranker	sharp 2nd-stage judge (“retrieve 50, rerank to 5”)	<code>config.RERANKERS</code>	<code>providers/rerank.py</code>

Table 1: The three swappable axes. Each is a registry dict in `config.py` read through a thin adapter; the stages and the app never import a vendor SDK.

5.1 The pattern (the same for all three)

Internalise this once and every future model plugs in the same way: a single public function (`generate` / `embed_texts` / `rerank`) looks up `provider` in the config dict and dispatches to a small per-provider implementation. Each implementation is the *only* place that imports a vendor SDK, so adding a Claude or OpenAI model is one new branch next to the Gemini one — the rest of the system doesn’t notice.

```
def generate(prompt, model=None, ...):
    model = model or config.DEFAULTS["llm"]
    prov = config.LLMS.get(model, {}).get("provider", "gemini")
    if prov == "gemini":
        return _gemini(prompt, model, ...) # the ONE place google.genai is imported
```

```
raise NotImplementedError(f"provider '{prov}' not wired yet")
```

5.2 The embedding axis has one extra twist: partitioned spaces

The vector store (`lib/vectorstore.py`) is **partitioned by space**, where a *space* is `model@dim` (e.g. `gemini-embedding-001@1536`). Each space gets its own folder under `data/vectors/<space>/` with `vectors.npy` + `ids.json` + `metas.jsonl`. RAG selects *one* space at query time, so you can populate several side-by-side and A/B them on the eval set without rebuilding anything. Adding a space is a one-line edit to `config.EMBEDDING_MATRIX`; the embed stage goes stale and re-embeds *only the new space*.

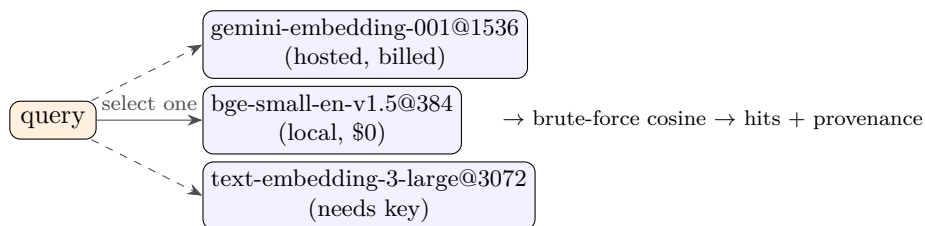


Figure 3: The vector store keeps one partition per *space*. The query picks a space (a variable); the others sit ready for an A/B without a rebuild.

6 Always-on cost logging

There is exactly **one funnel** for spend, and the rule is absolute: *no module ever calls a provider directly*. Every adapter call ends with `costlog.log(...)`, which appends a timestamped row to `costs/usage.csv`. If `usd` is omitted it is *estimated* from the price tables in `config.py`.

Two consequences fall out of this single design choice:

1. **The ledger is complete by construction.** The only path to a provider runs through an adapter, and every adapter logs — so there is no way to spend money invisibly. `costlog.summary()` aggregates the CSV by model into the running \$ total.
2. **The free path is genuinely free.** Local embeddings (`fastembed`), BM25, and the local cross-encoder reranker all log with `usd=0.0`. You can develop and run the whole *retrieval* side end-to-end without spending a cent; only hosted-model calls are billable, and they land in the same ledger.

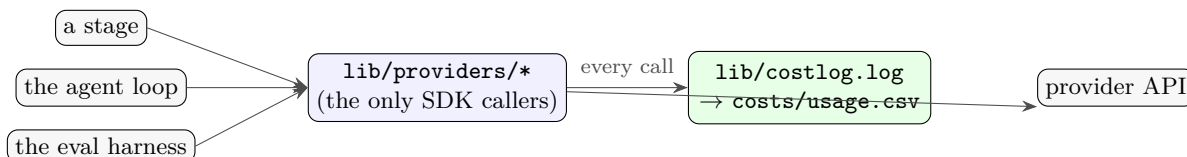


Figure 4: Every caller routes through an adapter; every adapter logs. The cost ledger cannot have a hole, and \$0 local calls are logged the same way as billed ones.

The adapters also **self-throttle**: bulk loops are wrapped in `tenacity` retry-with-backoff that treats `429` / `RESOURCE_EXHAUSTED` / `503` / `5xx` as transient and waits-and-retries exponentially.

A long run bends to the provider’s rate limit instead of crashing partway — the same lesson the OCR step learned. (For true bulk, the plan is the Gemini Batch API: –50%, no per-minute cap; gated until David says go.)

7 Provenance and archival standards

An archive’s whole value is *trust*: a claim you can’t trace is worthless to a historian. So provenance is threaded through every layer.

- **Chunks** carry {doc_id, rid, page, pdf_page, ...} from the moment they’re built.
- **The vector store** keeps that metadata beside every vector, so a hit already knows where it came from.
- **The knowledge graph** attaches source rids to every node and edge — so even an *inferred* fact points back at the regions that justify it.
- **Answers** cite doc_id + rid + page (+ vertices); the app’s citation modal turns those vertices into a **deep-zoom box on the page image** (IIIF Presentation 3.0 manifests + Web Annotations) and offers the source PDF to download.

On the standards side: the graph aligns to **RiC-O** (ICA Records in Contexts, proven on scholarly correspondence) and exports RDF/JSON-LD; dates use **EDTF** (the Library of Congress standard for uncertain/approximate/partial dates — exactly the “c. 1929” / “Mar. 30”-with-no-year mess this corpus is full of); entities are reconciled to **Wikidata** / **VIAF** / **GeoNames** / **Getty TGN** for global IDs; observability rides on **OpenTelemetry** / **Phoenix**.

8 The tiers (and the stages that fill them)

The build is organised into additive tiers (from RESEARCH_PLAN.md). Each raises the ceiling, and each new capability becomes a *toggleable tool* in the dev app — so you can measure exactly what each one buys you.

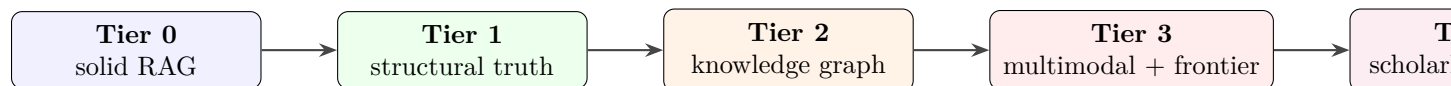


Figure 5: The recommended path. Each tier is additive; eval gates each step.

Tier 0 — solid RAG. `embed_corpus` → the vector store → `lib/retrieval.py` (router → query transforms like HyDE → hybrid BM25 + dense fused with RRF → cross-encoder rerank → optional Self-RAG/CRAG grading) → cited generation. This alone is a strong archival search/QA tool.

Tier 1 — structural truth. The corpus problems, built first because everything downstream depends on getting the *units* right:

- **stitch_notebooks** (A1) — reconstruct *logical* entries across “Page N Cont.” breaks, so a favour split over two pages is retrieved (and cited) as one whole thought.

- **normalize_dates** (A2) — combine the entry’s month/day with the *page*’s archival year, carry forward the last-seen date, capture *bitemporal* enrolled-vs-reported pairs → EDTF. Free rule path; gated LLM normaliser.
- **contextualize_chunks** — Anthropic-style *Contextual Retrieval*: prepend a 1–2 sentence situating preamble to each chunk *before* embedding/BM25, so the bare “22 Cured.” entry stops being invisible. The highest-leverage retrieval upgrade for *this* corpus.
- **extract_entities** (NER) — two passes: a *free* structured harvest (the labelled fields are entities we already know the type of), then a gated LLM pass over the prose for rich nested entities.
- **resolve_entities** (A3) — the classic ER pipeline (normalise → block → match → cluster → canonicalise → authority-reconcile) so “Grace.”, “Grace Panyard”, and “Mrs. Grace Panyard” collapse to one authority record.

Tier 2 — knowledge graph + GraphRAG. **build_graph** — a temporal property graph (networkx) aligned to RiC-O, exported to RDF; community detection + summaries for thematic questions. **index_sources** — a SQLite **FTS5** full-text index + **IIIF** manifests + text-layer searchable PDFs (all free, all local).

Tier 3 — multimodal + frontier. **colpali_index** — visual retrieval over the *page images* with late interaction (catches what the OCR missed), fused with text. **raptor** — a recursive cluster-and-summarise tree for global/thematic questions.

Tier 4 — scholarly platform. IIIF deep-zoom citations, Linked-Data exports (RiC-O / EDTF / authority IDs), fine-tuned models (deferred per David), hosting + drift monitoring.

Current state

embed_corpus and **normalize_dates** are wired into `run.py` and the base embeddings are built (`gemini-embedding-001@1536` and a free local `bge-small-en-v1.5@384`). The remaining stages exist as substantial drafted modules and become live as they are integrated and tested. The DAG is fully *inspectable* today (`python run.py graph`); paid stages stay gated until explicitly run.

9 Where the axes and tiers meet the user: the dev app

The capabilities surface in a FastAPI app (`app/server.py`) that exposes a **tool-using agent**: given a question plus the three model variables and a set of *enabled tools*, it runs a Gemini function-calling loop over *only those tools* and returns `{answer, citations[], trace[], grade, cost}`. Each capability is a plugin in `app/tools/` (`vector_search`, `pdf_fulltext_search`, `graph_query`, `entity_lookup`, `temporal_query`, `community_summary`), so it shows up as a *toggle* in the UI — which is precisely how you measure what each tier contributes. The server never calls a provider directly; every turn routes through `lib/providers/*` → `lib/costlog`, so the ledger stays complete and the free path stays \$0.

The `eval/` harness closes the loop: a set of **golden questions** (each with `ground_truth`, `expected_where`, `relevant_doc_ids`) is run through the real stack and scored on the RAGAS/ARES metrics (faithfulness, context precision/recall, answer relevance) for *every combination* of the three axes — so “which config to ship” is an empirical read-off, not a guess.

10 Recommendations: how to extend

The whole point of the architecture is that extension is *local*. The four common moves:

- **Add a stage.** Write a function in `stages/`, then register a `Stage(name, fn, deps, inputs, outputs)` in `run.py`. The DAG handles staleness and ordering — you never write scheduling code.
- **Add an embedding space.** Add a `(model, dim)` tuple to `config.EMBEDDING_MATRIX` (and the model to `config.EMBEDDINGS` if new). The embed stage re-embeds *only the new space*.
- **Add a model.** Add it to `config.LLMS` / `EMBEDDINGS` / `RERANKERS` with its price, and add the one provider branch in the matching adapter. Stages reading `DEFAULTS` pick it up with no edits.
- **Add an agent tool.** Drop a module in `app/tools/`; it appears as a toggle in the UI and a function declaration to the LLM.

Rule of thumb: if you find yourself editing a *stage* to swap a model, stop — that’s a sign the variable belongs in `config.py` instead.

11 Caveats

Read before you run anything billed

- **Prices in `config.py` are placeholders.** They let `costlog estimate` spend; verify each against the provider’s current pricing page before any billed run. Nothing is billed until a stage is actually executed.
- **Several stages are drafted, not yet live.** The DAG is inspectable now; integration + small smoke-tests gate each paid stage. `NotImplementedError` is the honest placeholder.
- **Fingerprints hash *file contents*, not semantics.** A whitespace-only edit to a stage’s source *will* mark it stale (it changes `inspect.getsource`). That is the conservative, correct default — better to re-run than to silently serve a stale artifact.
- **The vector store is brute-force cosine.** Instant at ~9k vectors, deliberately so; swappable for LanceDB/FAISS later *without changing callers* if the corpus grows.
- **Heavy/paid steps are deliberately deferred.** ColPali (vision model + GPU) is scaffolded but its heavy index build is flagged; full LLM NER and full multi-space embedding run only after a small cost-scoping batch, and only when David says go.

12 Sources

In-repo. `README.md` (design commitments + milestones); `RESEARCH_PLAN.md` (techniques, tiers, research basis); `config.py` (the three axes + prices); `lib/pipeline.py` (the DAG engine); `lib/costlog.py` (the cost funnel); `lib/chunks.py` / `lib/vectorstore.py` (data shapes + the partitioned store); `lib/providers/{llm,embed,rerank}.py` (the adapters); the `stages/` modules and the `app/` server + tools.

External (from RESEARCH_PLAN.md). Anthropic Contextual Retrieval; RiC-O (ICA Records in Contexts); EDTF (Library of Congress / ISO 8601-2); Splink (probabilistic record linkage); RAPTOR (Sarathi et al. 2024); ColPali (late-interaction visual retrieval); RAGAS / ARES (RAG evaluation); OpenTelemetry / Arize Phoenix (observability).